

Unidad 1 - Ingeniería de software

Introducción a la ingeniería del software

Software:

El software es un conjunto de programas en conjunto con toda la documentación requerida para definir, desarrollar y mantener los programas ejecutables que se entregan al cliente, como así también las herramientas utilizadas para la construcción de este.

Es la información o conocimiento que se presenta en distintos niveles de abstracción, desde los requerimientos del sistema (abstracto), hasta el código que ejecuta el mismo (detallado).

Ingeniería de Software:

Es una disciplina que se encarga de todos los aspectos de la producción del software, desde la primera etapa de especificación del sistema hasta el mantenimiento del sistema después de que se pone en operación.

Esta ingeniería es importante ya que:

- Con mayor frecuencia se requiere producir de manera rápida y económica sistemas confiables.
- Resulta más barato a largo plazo utilizar métodos y técnicas de ingeniería de software para los sistemas de software. La mayoría de los costos consisten en cambiar el software después de que se pone en operación.

Además, la ingeniería de software se apoya en varias realidades, entre ellas:

- Se debe hacer un esfuerzo para entender el problema antes de desarrollar una aplicación de software.
- El diseño es una actividad crucial en el desarrollo de un software.
- Realizar un correcto diseño trae consigo calidad de software y facilidad de mantenimiento.

La ingeniería de software nace tras las crisis del software, en la cual existía (y existe) un conjunto de dificultades o errores ocurridos en la planificación, estimación de los costos, productividad y calidad de un software, debido, principalmente, a la baja eficacia que presentan una gran cantidad de empresas al momento de desarrollarlo.

Estado actual y antecedentes - La crisis del software

Causas: La evolución de la tecnología del hardware mediante los circuitos integrados permitió el desarrollo de sistemas más grandes y complejos. El salto en el hardware no fue acompañado por un salto en el desarrollo del software, esto en gran parte se debe a que el software es una actividad humana cuyo producto es abstracto e intangible.

En combinación con lo anterior, la demanda creciente de estos sistemas complejos, la subestimación de la complejidad que supone el desarrollo de software, los cambios a los que tienen que ser sometidos los productos para ser adaptados a las necesidades del cliente y la falta de una disciplina que intervenga en todos los aspectos de la producción del software fueron las causas de esta crisis.

En la actualidad, muchos de los sistemas desarrollados fracasan debido a fallas en el software, que principalmente son consecuencia de dos factores:

- Demandas crecientes: a medida que la tecnología y las técnicas de desarrollo evolucionan, la demanda de sistemas más grandes y complejos aumenta. La problemática aquí es que los métodos tradicionales de ingeniería no permiten lograr esta “espontaneidad”.
- Bajas expectativas: muchas de las compañías actuales de desarrollo de software no utilizan métodos de ingeniería de software en su trabajo diario. Por lo tanto, su software con frecuencia es más costoso y menos confiable de lo que debiera. La consecuencia es que las empresas se conforman con que el software solo “funcione”, sin tener en cuenta el valor agregado que el software le debe aportar al negocio del usuario.

Disciplinas que conforman la ingeniería de software

- Disciplinas técnicas: actividades que aportan al desarrollo de software como producto:
 - Toma de requerimientos
 - Análisis de requerimiento
 - Diseño de software
 - Implementación de software
 - Prueba de software
 - Despliegue de un software
 - Documentación de software
 - Capacitaciones a usuarios
- Disciplinas de gestión: actividades de planificación, monitoreo y control del proyecto de desarrollo de software. Se utilizan, por ejemplo, metodologías LEAN Agile de gestión de proyectos.
- Disciplinas de soporte: son transversales a los procesos de software, que permiten verificar la integridad y calidad de un producto de software. Entre ellas se incluyen:
 - Gestión de Configuración del Software (SCM)
 - Toma de métricas
 - Aseguramiento de calidad (QA - Quality Assurance)

Ciclos de vida (Modelos de Proceso) y su influencia en la Administración de Proyectos de Software.

Se denomina ciclo de vida a una representación simplificada de un proceso, el cuál define elementos del mismo (actividades estructurales, productos del trabajo, tareas, etc.) y el flujo del proceso (o flujo de trabajo), que especifica la relación y el orden de dichos elementos. Son modelos genéricos de los procesos de software; y establecen los criterios que utiliza para determinar si se debe pasar de una tarea a la siguiente. Y son independientes de los procedimientos de cada actividad del ciclo de vida.

Los modelos de proceso pueden incluir las mismas actividades, pero cada uno, desde una perspectiva particular, pondrá más énfasis en algunas respecto a otras y definirán flujos de procesos diferentes. Además, establece los criterios que se utilizan para determinar cuándo se procede de una tarea a otra. Los modelos de ciclos de vida especifican:

- Las fases del proceso (requerimientos, especificaciones, diseño, etc.)
- Orden en el cual se llevan a cabo.

Relación entre ciclo de vida del proyecto y del producto

No existe un impacto entre los ciclos de vida de cada uno, sino que el ciclo de vida del producto siempre es mayor que el ciclo de vida del proyecto, ya que el ciclo de vida del proyecto dura lo que dura el desarrollo del software. En cambio, el ciclo de vida del producto dura hasta que el software se deje de utilizar. Es posible que un producto tenga varios proyectos en su ciclo de vida, debido a las actualizaciones que se vayan realizando.

Clasificación de ciclos de vida

Modelo secuencial:

Este modelo dispone de las actividades de forma lineal, es decir, que el proyecto progresa a través de una secuencia ordenada de pasos (fases) y una actividad no puede iniciar sin que la precedente haya sido finalizada. El avance entre las fases se da tras una revisión al final de cada una de estas para determinar si el software está listo para avanzar.

Se suele utilizar en aquellos sistemas donde los requerimientos se comprenden bien y son exhaustivos ya que éstos se congelan al finalizar la etapa de toma de requerimientos.

Generalmente, este tipo de modelos está dirigido por documentos, es decir, el trabajo principal del producto es la documentación del software entre las distintas fases.

Modelo iterativo incremental:

Este modelo aplica sucesivas iteraciones en forma escalonada a medida que avanza el calendario de actividades. Cada iteración produce un incremento de software funcional potencialmente entregable. Usualmente los primeros incrementos incluyen las funciones básicas/críticas que más requiere el cliente. Este enfoque vincula las actividades de especificación, desarrollo y validación. Este ciclo de vida se utiliza en metodologías ágiles.

Ventajas frente al modelo secuencial

- Se reduce el costo de adaptar los reques cambiantes del cliente y el retrabajo es menor ya que en cada incremento se obtiene una retroalimentación.
- El cliente es parte del proceso de desarrollo.
- Es posible que la entrega rápida e implantación de software útil al cliente, aún si no se ha incluido toda la funcionalidad. Los clientes tienen la posibilidad de usar y ganar valor del software más temprano de lo que sería posible con un proceso en cascada.

Dificultades desde una perspectiva administrativa

- Alto costo de documentar cada incremento.
- La estructura del software tiende a degradarse frente a una cantidad considerable de incrementos.

Modelo recursivo:

El modelo recursivo es utilizado para gestionar los riesgos del desarrollo de sistemas complejos a gran escala. Requiere de la intervención del cliente. El modelo en espiral es un modelo recursivo, que consta de 4 etapas recursivas:

1. Determinar objetivos.
2. Identificar y resolver los riesgos.
3. Desarrollar y probar.
4. Planificar la próxima iteración.

Para llevarlo adelante, se subdivide el proyecto en proyectitos, intentando resolver en cada uno los riesgos más relevantes hasta que no quede ninguno. Se presenta un prototipo que va mejorando con cada vuelta. Lo positivo, es que se generan productos independientes de la implementación, que pueden ser reusables en sistemas de características similares.

Dificultades desde una perspectiva administrativa

- Puede ser más costoso en tiempo y dinero adaptarlos para utilizarlos para diferentes proyectos.
- Puede carecer de mantenimiento o documentación, lo cual dificulta mucho las tareas.
- Se genera una versión del producto recién al final.

Administración de proyectos dependiendo en ciclo de vida

La elección de un ciclo de vida afecta de forma directa a la administración que se debe realizar sobre el proyecto, ya que cada uno de los modelos de proceso poseen características y enfoques distintos. En este caso, la administración y la planificación del proyecto se ve afectada en la elección del ciclo de vida. Además, durante el desarrollo del proyecto, la gestión de este se realiza en consecuencia de esta elección, ya que la gestión de cada uno es distinta.

Procesos de desarrollo Empíricos vs Definidos

Proceso de desarrollo de software

Un proceso es una secuencia de pasos ejecutados para un propósito dado (IEEE), mientras que un proceso de software se define como un conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software.

Las actividades del proceso de desarrollo son la especificación de requerimientos, el análisis, el diseño, la implementación y la prueba. El orden y la relación entre las diferentes actividades está determinada por la elección del ciclo de vida.

Se tienen tres factores determinantes del proceso:

- **Procedimientos y Métodos:** La adaptación de los procesos establecidos para su implementación, dependiendo del contexto, es esencial para la calidad resultante. Definir las actividades que se harán y cuáles no. Además, es relevante que esta información esté documentada para lograr transparencia.
- **Personas motivadas, capacitadas y con habilidades:** Es factor primordial para obtener la calidad del producto del proceso, ya que éste se determina del esfuerzo de las personas. Sin ellas, no se puede lograr construir ningún producto.
- **Herramientas y Equipos:** Materiales necesarios para llevar a cabo el proceso que nos permiten que las entradas se transformen en salidas. Se recomienda automatizar la mayor cantidad de actividades posibles, lo que mejora la eficiencia de las personas y puedan concentrarse en tareas que requieran de su capacidad.

Los procesos de software son complejos, por lo que no hay un proceso ideal; además, la mayoría de las organizaciones han diseñado sus propios procesos de desarrollo de software. Los procesos han evolucionado para beneficiarse de las capacidades de la gente en una organización y de las características específicas de los sistemas que se están desarrollando. Para algunos sistemas, como los sistemas críticos, se requiere de un proceso de desarrollo muy estructurado (proceso definido). Para los sistemas

empresariales, con requerimientos rápidamente cambiantes, es probable que sea más efectivo un proceso menos formal y flexible (filosofía ágil).

Consideraciones

1. Estas actividades varían dependiendo de la organización y el tipo de sistema que debe desarrollarse, pero deben incluir: productos, roles, responsabilidades y condiciones
2. El proceso debe ser explícitamente modelado si va a ser administrado.
3. Un proceso de desarrollo se puede aplicar a varios ciclos de vida. La elección del ciclo de vida depende de la estrategia y este debe permitir seleccionar los artefactos a producir, definir actividades y roles, y modelar conceptos.
4. Las personas utilizan herramientas y equipos para llevar a cabo los procedimientos que componen el proceso de desarrollo.

Actividades fundamentales

1. Especificación de software: clientes junto con ingenieros definen el software a producir y sus restricciones.
2. Desarrollo: diseño y programación del software.
3. Validación: verificación para asegurar que es lo que el cliente quiere.
4. Evolución: donde se modifica el software para reflejar los requerimientos cambiantes del cliente y mercado.

Procesos definidos

Son considerados deterministas: Asumen que, si se aplican una y otra vez, se obtendrán siempre los mismos resultados, a pesar de cambiar de equipo.

1. Están inspirados en las líneas de producción.
2. La administración y el control provienen de la predictibilidad del proceso. Es decir, el objetivo es lograr una repetibilidad, para poder obtener una previsibilidad de esfuerzo, riesgos y costos asociados al desarrollo de software.
3. Puede usar cualquiera de los ciclos de vida.

En realidad, no es posible obtener el mismo resultado ante mismas entradas ni con el mismo equipo, ya que depende sumamente del contexto, momento y las personas, por más parecido sean estos factores.

La administración y control del proceso, en muchos de los casos, toman más importancia que el avance del software en sí (más importante la documentación del avance, que el avance en sí).

Procesos empíricos

Se conforman en base a la experiencia interna y externa de las personas que intervienen en un contexto particular. NO es considerado determinista ya que no se asume que ante la aplicación del mismo proceso se obtendrán los mismos resultados. Dado que el factor diferencial es la experiencia sensible, en estos procesos se pueden obtener diferentes resultados dependiendo del contexto en el cual se apliquen.

1. Se basan en ciclos cortos de retroalimentación, porque ante un análisis, se ajustan de mejor forma en aquellos dominios complejos donde prima la creatividad y/o complejidad.

2. La administración y el control es por medio de inspecciones frecuentes y adaptaciones para lograr buenas prácticas.
3. No se pueden combinar con cualquiera de los tipos de ciclos de vida vistos, se suele recomendar que sea el modelo iterativo-incremental, y el que se suele prohibir es el modelo secuencial debido a que no es viable después de 2 años (lo dice una teoría).

La experiencia que se obtiene en cada uno de los procesos no es extrapolable, es decir, que solo funciona en ese mismo proyecto, y se necesita adaptar los procesos a cada contexto en especial. Los conocimientos obtenidos en el proceso se deben hacer explícitos al grupo de desarrollo, y convertir las habilidades individuales en buenas prácticas del trabajo en grupo (compartir conocimiento). Además de la inspección y adaptación, los procesos empíricos poseen un tercer pilar muy importante, que es la transparencia de la información ante todos los integrantes del equipo de trabajo.

Ciclos de vida (Modelos de Proceso) y Procesos de Desarrollo de Software

Ciclo de vida	Procesos de desarrollo que lo admite	Características que lo hacen elegible
Secuencial	Definidos	- Baja incertidumbre - Org. con forma trad. de trabajar - Se debe entregar todo el software junto - Eliminar riesgos de planificación
Iterat./Inc.	Definidos o empíricos	- Existe incertidumbre - Volatilidad de requerimientos - Posibilidad de entregas parciales - Uso real en etapas tempranas
Recursoivo	Definidos	- Mucho control de riesgos en cada iteración - No se pueden realizar entregas parciales

Diferentes modelos de ciclo de vida

Code and Fix

Se desarrolla sin especificaciones o diseño y se modifica hasta que el cliente esté satisfecho. Los cambios se realizan durante el mantenimiento, lo que es muy caro.

Este modelo tiene dos ventajas:

- No se dedica tiempo a planificar, documentar, hacer control de calidad o cualquier actividad que no sea codificación pura. Es decir, se salta directamente desde un principio a la codificación y los progresos se ven inmediatamente.
- Requiere poca experiencia, cualquiera que sepa un lenguaje de programación puede familiarizarse con este modelo.

Puede ser útil para proyectos pequeños que se tiene la intención de desecharlos después de construirlos

Modelo de cascada puro

Se sigue una secuencia de pasos ordenada y se realiza revisión al final de cada fase. Es conducida por la documentación con fases que no se superponen.

- Permite encontrar errores en etapas tempranas.
- Hay exceso de documentación y falta de resultados hasta el final.
- Se utilizan cuando la definición del producto es estable o los requerimientos de calidad dominan a los costos y tiempos.
- Funciona bien cuando se tiene un equipo técnicamente débil o con poca experiencia porque provee al proyecto una estructura que ayuda a minimizar el esfuerzo desperdiciado.

Este modelo no es flexible, se requiere especificar completamente los requerimientos al comienzo y puede pasar mucho tiempo hasta que se tenga software funcionando. Olvidarse de algo puede ser un error muy costoso.

Modelo de cascada con fases solapadas

Hay un fuerte grado de solapamiento. La documentación es menor, pero es más difícil hacer el seguimiento de progreso.

Modelo de entrega por etapas

Se ven signos tangibles de progreso. Es útil cuando el cliente necesita funcionalidad de inmediato y las necesidades se modifican. Debe haber una planificación rigurosa para que funcione.

Modelo en cascada con retroalimentación

Es una variación del modelo clásico en que es posible volver a una etapa anterior antes de llegar al final, aunque es muy caro hacerlo.

Modelo en cascada con subproyectos

Es un modelo secuencial. Se avanza en cascada hasta el diseño arquitectónico, de ahí en más se avanza en partes dividiendo en Subproyectos.

Modelo en espiral

Busca minimizar los riesgos haciendo un análisis de riesgos y buscando alternativas al iniciar cada fase. Al final de cada fase se planifica la siguiente y se evalúa si se sigue adelante. Este modelo parte el proyecto en mini proyectos que permite manejar mejor los riesgos.

- Permite evaluar la factibilidad de un nuevo producto.
- En proyectos grandes la identificación de los riesgos puede costar más que el desarrollo.
- Es un modelo adecuado para proyectos de vida largos, que puede utilizar equipos diferentes en cada ciclo y muestra el progreso al final de este.
- Por lo general a medida que el costo incrementa, el riesgo disminuye.
- Puede ser difícil definir cuándo se está listo para avanzar a la siguiente capa del espiral.

Modelo evolutivo

Se fija el tiempo o el alcance mientras el otro se va modificando. Es útil cuando los requerimientos no son claros y se realizan entregas tempranas al cliente. El problema es

que es un modelo recursivo que repite todas las tareas y depende de que los diseñadores desarrollen un sistema fácil de modificar.

Diseño para cronograma

No se asegura alcanzar la versión final, aunque sí una entrega del producto para la fecha definida con los aspectos más importantes. Es útil cuando no hay seguridad sobre las capacidades de programación.

Diseño para herramientas

Consiste en incluir solo la funcionalidad soportada por las herramientas de software existentes. Se usa en proyectos muy sensibles al tiempo. Puede combinarse con otros ciclos de vida, pero no se podrán implementar todos los requerimientos.

Prototipación evolutiva

Se da cuando hay cambios rápidos de requerimientos y los clientes no se comprometen con los requisitos. Requiere menos documentación, pero no se pueden programar los release. Se desarrolla el concepto del sistema a medida que se avanza. Es útil cuando los desarrolladores no están seguros de que algoritmos o arquitectura óptima utilizar.

Componentes de un proyecto de sistemas de información

Proyecto

Un proyecto de software es un esfuerzo temporal que requiere del acuerdo de un conjunto de especialidades y recursos para la creación de un producto, servicio o resultado único. Define el proceso y tareas que se van a realizar, el personal que se encargará de las actividades y los mecanismos que se implementarán para valorar riesgos, controlar el cambio y evaluar la calidad.

Características

Únicos: El resultado de un proyecto será único e irreplicable para cualquier otro proyecto, por más parecido o igual sean sus elementos componentes.

No ambiguos: deben ser lo suficientemente claros para guiar el desarrollo del proyecto.

Medibles: es necesario medir el objetivo para determinar el avance sobre el proyecto.

Alcanzable: deben ser factibles de realizarse por el equipo (que el equipo pueda hacerlos).

Duración limitada de tiempo: Esto implica que un proyecto tiene un principio y un fin, liberando los recursos y personas.

Conjunto de tareas interrelacionadas basadas en esfuerzo y recursos: Se definen las tareas, sus dependencias, los recursos y personas asignadas, permitiendo manejar la complejidad inherente de los sistemas.

Administración de un proyecto de software

Es la organización y coordinación de personas y recursos que permiten desarrollar un producto o servicio de acuerdo con el presupuesto, tiempo y funcionalidades acordadas con el cliente, asegurando que se entregue un producto o servicio de alta calidad. Esta disciplina tiene dos grandes actividades: Planificación y Monitoreo y Control.

La administración incluye:

1. Identificar los requerimientos.
2. Establecer objetivos claros y alcanzables.
3. Adaptar las especificaciones, planes y el enfoque a los diferentes intereses de los involucrados (stakeholders)

Una buena gestión no puede garantizar el éxito del proyecto. Sin embargo, una mala gestión usualmente implica una falla en el proyecto.

Metas de la administración de proyectos:

1. Entregar software al cliente en el tiempo acordado.
2. Mantener los costos dentro del presupuesto general.
3. Entregar software que cumpla con las expectativas del cliente.
4. Mantener un equipo de desarrollo óptimo y con buen funcionamiento

Atributos particulares o complejidades en la administración de proyectos de software:

1. El producto es intangible: No poder ver ni tocar el producto que se obtiene al ejecutar el proyecto dificulta la medición del progreso del proyecto.
2. Los proyectos de software suelen ser excepcionales: Los conocimientos aprendidos en un proyecto de software son difícilmente extrapolables a otro proyecto. El factor tecnológico es una de las causas.
3. Los procesos de desarrollo son variables y específicos de la organización: A pesar del intento de estandarizar los procesos de desarrollo, la realidad es que no es posible predecir de manera confiable qué proceso es el adecuado para adoptar en el proyecto.

La triple restricción - enfoque tradicional

Las variables de restricción de un proyecto son 3: tiempo, costo y alcance. El balance de estos tres factores afecta directamente la calidad del proyecto, ya que un proyecto de alta calidad es aquel que entrega el producto, servicio o resultado requerido, satisfaciendo los objetivos en el tiempo estipulado y con el presupuesto planificado. Es responsabilidad del Líder de proyecto balancear estas variables.

El concepto de la triple restricción hace referencia a que no es posible fijar de forma arbitraria a las 3 variables, sino que será posible fijar 2 y la tercera se ajustará a lo definido. No se negocia la calidad, si no puedo modificar tiempo y/o costo, debido a que es como entregar un software sin probarlo. La calidad del producto que le entrego al cliente se ve afectada cuando no puedo equilibrar estos 3 factores.

Alcance - ¿Qué trata de alcanzar?

Son los requerimientos que se incluyen en el proyecto y definen la funcionalidad que tendrá el producto a entregar al cliente. Esta variable es la que primero se negocia con el cliente.

Tiempo - ¿Cuánto tiempo me llevaría?

Define la fecha en la cual se compromete a finalizar el proyecto y entregarle el producto resultante al cliente. En las metodologías ágiles, el tiempo es fijo.

Costo - ¿Cuánto debería costar?

Define el costo de todos los recursos implicados en el desarrollo del proyecto. Esto incluye equipamiento, infraestructura, equipos, salarios, entre otros.

En la realidad, el costo y el tiempo del proyecto varían de forma directa con la definición del alcance del producto, es decir, el alcance es directamente proporcional al tiempo y al costo.

Líder de proyecto

En un enfoque tradicional, el líder de proyecto realiza la toma de decisiones en su totalidad sin la inclusión de la opinión del equipo. El líder asigna las tareas que deben realizar los integrantes del equipo, y además, maneja todas las relaciones con todos los stakeholders del proyecto (clientes, gerencias, contratistas, stakeholders en general).

Usualmente los profesionales técnicos no disponen de habilidades necesarias para tratar con la gente, por lo que es necesario una figura de líder de proyecto con las siguientes aptitudes:

1. Motivación.
2. Organización.
3. Innovación.
4. Empatía.
5. Comunicación.
6. Liderazgo.

También cuenta con habilidades duras, relacionadas a los conocimientos del producto, técnicas y herramientas. Dado que el líder de proyecto tiene un enfoque basado en la resolución de problemas, también son características propias las siguientes:

1. Resolución de problemas: identifica conflictos, estructura la solución o motiva a otros profesionales para que la desarrollen, aplica lecciones aprendidas en otros proyectos y se adapta ante inconvenientes.
2. Identidad administrativa: tiene la confianza de asumir el control cuando es necesario.
3. Logro: recompensa las iniciativas y los logros.
4. Influencia: define la estructura del equipo en función de la retroalimentación que él mismo suministra mediante palabras y gestos.

Entre las responsabilidades del líder de proyecto se encuentran, en las nuevas metodologías:

1. Definir el alcance del proyecto.
2. Identificar a los involucrados, recursos y presupuesto.
3. Detallar las tareas, estimar los tiempos y requerimientos.
4. Identificar y evaluar riesgos.
5. Preparar planes de contingencia.
6. Controlar hitos y participar en las revisiones de las fases del proyecto.
7. Administrar el proceso de control de cambios.
8. Producir reportes de estado.

Equipo de proyecto

Es un grupo de personas comprometidas con alcanzar un conjunto de objetivos de los cuales se sienten mutuamente responsables. Entre sus características se encuentran:

1. Cuentan con conocimientos, habilidades técnicas, motivación, experiencias y diversas personalidades.
2. Usualmente son un grupo pequeño, lo que permite una comunicación efectiva.
3. Trabajan en forma conjunta con espíritu de grupo.
4. Sentido de responsabilidad como una unidad y se focalizan en el cumplimiento de las metas grupales.

En equipos cohesivos:

1. El grupo establece sus propios estándares de calidad.
2. Los individuos aprenden de los demás y se apoyan mutuamente.
3. El conocimiento se comparte.
4. Prevalece la mejora continua.

Stakeholders

Son los interesados del proyecto, incluye el equipo de proyecto, el equipo de dirección, el líder de proyecto y el patrocinador.

Plan de proyecto

El plan de proyecto es un artefacto de gestión que cumple la función de “hoja de ruta” de un proyecto, en la cual se documentan todas las decisiones que se toman a lo largo del desarrollo.

En un plan de proyecto se establece qué se va a hacer (alcance del proyecto), cuándo se va a hacer (calendario), cómo se va a hacer (actividades o tareas para cubrir el alcance) y quién lo va a hacer (responsables de cubrir las actividades). Responder a estas preguntas, implica las siguientes actividades:

1. Definición del alcance del proyecto

El alcance de un proyecto responde al qué se está desarrollando. Es importante diferenciar el alcance de un producto y el del proyecto en sí. El alcance de un producto define qué funcionalidades debe realizar el software para satisfacer los requerimientos del cliente y se mide contra la Especificación de Requerimientos (ERS), en cambio, el alcance del proyecto define solo el trabajo necesario que se debe realizar para cumplir con el desarrollo de este y poder entregar el producto o servicio con todas las características y funciones especificadas.

Tanto el alcance como el objetivo de un proyecto puede ser modificado a lo largo del proyecto, pero esto no necesariamente tiene que cambiar el alcance del producto. Por otra parte, el alcance de un producto sí condiciona los alcances de un proyecto.

Ejemplos de alcances de un proyecto, pueden ser el realizar la toma de requerimientos, realizar Testing de determinados componentes, hacer el desarrollo del código de un componente, etc.

2. Definición del proceso y ciclo de vida

Al definir el proceso que se va a utilizar, se responde al cómo se va a desarrollar el proyecto.

Se debe tener en cuenta la información del contexto de desarrollo y la complejidad del software a desarrollar (alcances y objetivos).

El ciclo de vida del proyecto es lo que define cuánto de cada tarea se debe hacer y en qué momento del desarrollo del proyecto hacerlo. El ciclo de vida de un proyecto define lo siguiente:

1. Qué trabajo técnico debería realizarse en cada fase.
2. Quién debería estar involucrado en cada fase.
3. Cómo controlar y aprobar cada fase.
4. Cómo deben generarse los entregables.
5. Cómo revisar, verificar y validar el producto.

En metodologías tradicionales, se permite elegir cualquier tipo de ciclo de vida (cascada, iterativo e incremental, espiral, etc.), en cambio en metodologías ágiles esto no es posible (**sólo** iterativo e incremental).

3. Estimación

En un proyecto de desarrollo de software, se trata de estimar para predecir el valor de un elemento relacionado con el proyecto o con el producto. En un enfoque tradicional, esta estimación de valores es realizada únicamente por el líder de proyecto. En la metodología tradicional, existe un orden definido y recomendado para realizar las estimaciones:

1. Tamaño del producto: Se utilizan los requerimientos tomados en la etapa de requisitos. El aumento o disminución del tamaño del producto afecta de gran manera a otros aspectos, ya que un aumento del tamaño trae consigo un aumento de complejidad del software, que puede repercutir de muchas maneras en el desarrollo. La mayoría de las veces, el error de las estimaciones recae en estimar costos, tiempos y esfuerzo sin tener una estimación del tamaño, y luego en el transcurso del desarrollo, no variar este costo, tiempo y esfuerzo si varía el tamaño del producto (gran desventaja de la metodología tradicional → no responder a cambios de forma flexible).
2. Esfuerzo: Intenta responder cuántas horas personas lineales serán necesarias para construir el producto, previamente estimado su tamaño. El esfuerzo varía directamente con el tamaño de un proyecto. Cada rol puede tener un valor de hora distinto o bien, se puede realizar una estimación en horas planas, en donde no se hace distinción de valor por rol. El esfuerzo es el tópico que más repercute en el costo del proyecto, abarcando casi el 80% de estos. Además, el efecto de las habilidades personales de los integrantes del grupo define el esfuerzo final del proyecto.
3. Calendario: Responder a cuándo se van a realizar las tareas definidas. En el momento de la estimación, no es necesario realizar una calendarización muy específica a nivel de día, sino que se escala a tareas a realizar en, por ejemplo, un mes del año o incluso un trimestre. Es necesario ya tener una buena estimación del tamaño y del esfuerzo que se requiere, ya que el tiempo de duración del proyecto es proporcional a su crecimiento.
4. Costos: Depende directamente del tamaño y del esfuerzo que se necesite para el desarrollo del software, por eso es lo último que se estima.

4. Gestión de riesgos

Un riesgo es la probabilidad de ocurrencia de una pérdida o daño que afecte de forma directa al proyecto y pueda comprometer el incumplimiento de su objetivo. Es de suma

importancia identificar y definir los riesgos a los que está atado un proyecto, para así poder analizarlos, y realizar una cuantificación de estos, teniendo en cuenta la exposición del proyecto ante los mismos, para así definir a cuáles se debe atender y sobre cuáles se debe hacer un seguimiento.

Los podemos clasificar en:

1. Riesgos del proyecto: aquellos que alteran el calendario o los recursos del proyecto. Un ejemplo de riesgo de proyecto es la renuncia de un diseñador experimentado.
2. Riesgos del producto: aquellos que afectan la calidad o el rendimiento del software a desarrollar. Un ejemplo de riesgo de producto es la falla que presenta un componente que se adquirió al no desempeñarse como se esperaba.
3. Riesgos organizacionales o de negocio: aquellos que afectan la calidad o el rendimiento del software a desarrollar. Un ejemplo de riesgo de producto es la falla que presenta un componente que se adquirió al no desempeñarse como se esperaba.
4. Riesgos organizacionales o de negocio: aquellos que afectan a la organización que desarrolla o que adquiere el software. Por ejemplo, un competidor que introduce un nuevo producto.

Para medir los riesgos se utiliza lo que se conoce como exposición al riesgo, la cual se obtiene como el producto entre la probabilidad de ocurrencia de la amenaza por el impacto que generaría si realmente ocurriera. Este valor permite organizar y gestionar los riesgos, y se debe hacer foco en reducir la exposición del riesgo.

Antes los riesgos identificados, es posible tomar 3 actitudes:

- Negación: hacer de cuenta que los riesgos no existen.
- Gestión Reactiva: esperar a que el riesgo ocurra y recién ahí ver que hacer.
- Gestión Proactiva: ir trabajando sobre el riesgo generado planes de mitigación y de contingencia.

Al tomar una actitud proactiva, la identificación y análisis de estos riesgos, permite definir para cuáles de esos riesgos identificados es importante definir planes de mitigación y contingencia. La Identificación y análisis permite reconocer los riesgos y analizar su importancia, luego los planes de mitigación permiten bajar la exposición del proyecto ante ese riesgo identificado y analizado. Por último, los planes de contingencia permiten actuar en caso de que un riesgo suceda, y poder volver a la normalidad lo antes posible.

Gestión del riesgo

1. Identificación de riesgos: se identifican los riesgos que suponen una mayor amenaza al proceso de ingeniería de software, al software a desarrollar o a la organización que lo desarrolla (al recibir los requerimientos).
2. Análisis de riesgos: para cada riesgo identificado se realiza un juicio acerca de la probabilidad y del impacto. Se utilizan intervalos de probabilidad y clasificaciones de gravedad. El criterio dependerá de la experiencia de quien realice el análisis de riesgos.
3. Planeación del riesgo: para cada riesgo analizado, se definen estrategias para manejarlos. Estas estrategias consisten en considerar acciones a tomar para minimizar o evitar el impacto sobre el proyecto como consecuencia de la ocurrencia de la amenaza que representa el riesgo. Otras estrategias consisten en desarrollar

planes de contingencia, el cual define un plan para enfrentar una situación controversial.

4. Monitorización del riesgo y control: proceso que permite determinar que las suposiciones acerca de los riesgos de producto, proceso y organización no han cambiado. Esto permite revalorar al riesgo en términos de posibles variaciones de su probabilidad e impacto. Este proceso se aplica en todas las etapas del proyecto.

5. Asignación de responsabilidades (o recursos)

La asignación de responsabilidades a personas, permite asignar roles a los integrantes del equipo de trabajo. En el entorno de procesos definidos (PUD, por ejemplo), estos roles se ven definidos en el concepto de trabajadores. Dentro de un mismo proyecto, una persona puede desenvolverse en más de un rol.

Para dejar explícitos los roles y quienes son las personas que los cumplen, generalmente se construye una tabla, la cual contiene la información de la persona, el rol y responsabilidades de esta en el contexto del proyecto. Se debe hacer mucho foco en la selección del personal de un proyecto, ya que las capacidades de los integrantes del equipo afectan a la calidad del software y al correcto desarrollo del proyecto en términos de tiempo, por ejemplo, ya que el desarrollo de software es una actividad HUMANO-INTENSIVA.

6. Programación de proyectos (calendarización)

En esta etapa se trata de definir en detalle, cada tarea que debe ser cumplida en el desarrollo. Se toma como base lo definido en la estimación del calendario realizada previamente, pero se refina al máximo detalle posible cada actividad.

Cuando se habla de detallar las tareas, se hace referencia a descomponer el proceso de desarrollo en actividades refinadas a nivel de días, identificando quien la realiza, cuando debe realizarse y cuánto esfuerzo debe llevar en forma teórica.

Generalmente la calendarización se realiza a través de un Diagrama de Gantt, realizado a través de alguna herramienta, como Microsoft Project.

7. Definición de métricas

Las métricas se definen como una medida numérica que aporta visibilidad sobre el avance o estado del proyecto, proceso o producto en un momento determinado del desarrollo (representativas para el entorno). Al realizar la planificación del proyecto, es necesario definir de forma clara y no ambigua, cada una de las métricas asociadas a cada dominio en conjunto con la forma de calcularlas.

Las métricas del proyecto se consolidan para crear métricas de proceso que sean públicas para toda la organización del software.

A diferencia de las metodologías ágiles, acá se definen métricas que miden y exponen los avances. En cambio en ágil, el mayor indicador de avance de un producto, son los incrementos que se entregan en cada iteración.

8. Definición de controles

En la planificación de los controles del desarrollo, se toma como base las métricas ya definidas, y verifica que todo se está haciendo en concordancia con lo establecido. En esta planificación se definen reuniones periódicas, reportes o informes necesarios, etc.

Vínculo proceso-proyecto-producto en la gestión de un proyecto de desarrollo de software.



En el enfoque tradicional, previo a la definición del proyecto, es necesario determinar los objetivos y el ámbito (funciones y comportamientos que lo caracterizan) del producto para de esta manera poder realizar las estimaciones pertinentes.

Es necesario adaptar el proceso de desarrollo de software a cada proyecto, definiendo las actividades y métodos adecuados según sus características y complejidad. No existe un proceso ideal para todos los proyectos, por lo que debe utilizarse solo lo necesario para transformar los requerimientos en el producto o servicio esperado.

El proceso de desarrollo establece un marco y un plan para desarrollar el software. Las disciplinas de gestión adaptan las actividades a las características del proyecto, mientras que las disciplinas de soporte atraviesan todo el proceso. Primero se elige el modelo de proceso (secuencial, iterativo o recursivo) y luego se adapta el marco conceptual a dicho modelo.

Un proyecto de desarrollo está integrado por un equipo de trabajo, el cual va a basar su trabajo en el proceso adoptado, en el cual se van a tomar sólo aquellas actividades y formas de trabajo que deberán utilizar y seguir para lograr el desarrollo del producto/servicio deseado.

Las personas son la parte más importante, debido a que el desarrollo de software es una actividad humano-intensiva. Si se adopta el "mejor proceso" para el desarrollo de un software, y además se destina mucho esfuerzo a planificar el proyecto de desarrollo, de nada servirá si no tenemos un equipo capacitado que reúna las habilidades y herramientas necesarias para el desarrollo del mismo.

En el desarrollo de un proyecto, se busca utilizar diversas herramientas que se adapten y permitan automatizar o facilitar las actividades que están definidas en el proceso adoptado. Por ejemplo, el uso de un software de diseño para realizar el análisis y diagramación de clases.

No silver bullet

Brooks sostiene que no existe una "bala de plata" capaz de solucionar de golpe los grandes problemas de la ingeniería de software.

La metáfora viene del hombre lobo: así como una bala de plata permitiría eliminar al monstruo mágicamente, en software se busca una tecnología, lenguaje o metodología que produzca un salto enorme en productividad, calidad y simplicidad.

Su conclusión es que no existe esa solución mágica, porque una parte importante de la dificultad del software es inherente a su propia naturaleza.

Esencia vs Accidentes

Brooks divide las dificultades del desarrollo de software en:

- **Esencia:** Son las dificultades inherentes al problema que estamos intentando resolver. (Entender qué necesita el usuario, diseñar correctamente el sistema, etc.) Estas dificultades no desaparecen simplemente usando una tecnología mejor.
- **Accidentes:** Son dificultades producidas por la forma en que representamos o construimos el software, pero que no son inherentes al problema. (Trabajar con lenguaje de bajo nivel, tiempos de compilación y ejecución muy largos, etc.) Los avances tecnológicos han permitido reducir muchos de estos problemas.

El progreso histórico eliminó muchos problemas accidentales, pero ahora la mayor parte de la dificultad restante pertenece a la esencia del software. Por eso, seguir mejorando herramientas puede ayudar, pero no va a producir indefinidamente mejoras enormes.

Las cuatro dificultades esenciales

Brooks identifica cuatro características que hacen difícil al software:

1. **Complejidad:** El software tiene una enorme cantidad de elementos y relaciones entre ellos. Además, cuando crece, la complejidad puede crecer de manera más que lineal. Esto genera: dificultades de comunicación, errores, problemas de testing, etc. La complejidad es esencial, por lo que no podemos eliminarla simplemente con una herramienta.
2. **Conformidad:** El software debe adaptarse constantemente a cosas externas: otros sistemas, instituciones, reglas, interfaces, etc. Muchas de estas restricciones son arbitrarias y no pueden eliminarse simplemente modificando nuestro software.
3. **Cambio / modificabilidad:** El software cambia constantemente. Los usuarios quieren nuevas funcionalidades, aparecen nuevos dispositivos, sistemas operativos, necesidades, etc. Y como el software es relativamente fácil de modificar, existe todavía más presión para hacerlo. Por eso, un software exitoso inevitablemente termina cambiando.
4. **Invisibilidad:** El software no tiene una representación física directa. Un arquitecto puede mirar el plano de un edificio y visualizar su estructura. En software, en cambio, existen múltiples relaciones simultáneas: flujo de datos, control, dependencias, secuencias, relaciones entre componentes. No existe una única representación visual que capture toda esa complejidad.

¿Qué avances ya fueron importantes?

Brooks dice que algunos avances históricos sí produjeron grandes mejoras, pero porque eliminaron dificultades accidentales:

- **Lenguajes de alto nivel:** Fue uno de los mayores saltos de productividad, permitiendo trabajar con conceptos más cercanos al problema, en vez de bits y registros de memoria. Pero el beneficio no puede repetirse infinitamente: una vez eliminada la mayor parte de la complejidad accidental, queda la complejidad esencial.
- **Tiempo compartido:** Mejoró la productividad porque redujo los tiempos de espera y permitió al programador mantener mejor el contexto mental del sistema. Pero

también tiene un límite: cuando la respuesta del sistema ya es suficientemente rápida, reducirla aún más no genera grandes beneficios.

- Entornos integrados: La integración de herramientas, bibliotecas y formatos permitió reducir problemas derivados de utilizar muchas herramientas separadas.

¿Y las tecnologías que prometían ser la "bala de plata"?

Brooks analiza varias.

- Programación orientada a objetos: Tiene beneficios reales porque permite abstraer y modularizar mejor, pero no elimina la complejidad del problema que estamos diseñando. Solamente reduce parte de la complejidad accidental de expresar ese diseño.
- Inteligencia artificial: Brooks es escéptico respecto de que la IA pueda solucionar mágicamente el problema. Su argumento es que la dificultad principal está en decidir qué queremos decir/construir, no simplemente en encontrar una forma más fácil de expresarlo.
- Sistemas expertos: Sí los considera prometedores, especialmente para capturar y distribuir el conocimiento de programadores experimentados. Es un programa que intenta poner el conocimiento y experiencia de programadores expertos a disposición de otros programadores.
- Programación automática: La idea de escribir una especificación y que automáticamente se genere el programa parece revolucionaria. Pero Brooks señala que en muchos casos lo difícil no es escribir el programa, sino especificar correctamente qué debe hacer.

¿Cuál es entonces el camino que propone?

Refinamiento iterativo de requisitos + prototipación rápida

Brooks dice que: La parte más difícil de construir software es decidir qué construir. El cliente normalmente no sabe expresar completamente y con precisión lo que necesita antes de probar el sistema. Por eso propone trabajar de forma iterativa:

Requisitos → prototipo → feedback → refinamiento → nuevo prototipo → ...

El prototipo permite que el cliente vea y pruebe las ideas antes de construir el producto definitivo. Esto es importante porque ataca una dificultad esencial, no solamente una dificultad accidental.

Unidad 2 - Gestión Lean Agile de Productos de Software

La filosofía ágil busca entregar software frecuentemente y adaptarse rápidamente a los cambios en los requisitos y en el entorno. Para eso, el cliente participa activamente durante el desarrollo y proporciona retroalimentación continua.

Se basa en un desarrollo iterativo e incremental: el software se construye en pequeños incrementos, generalmente cada 2 o 3 semanas, incorporando nuevas funcionalidades en

cada uno. Esto permite inspeccionar el resultado y adaptar los requisitos según lo aprendido.

A diferencia de procesos más definidos como RUP, los métodos ágiles son adaptables en lugar de predictivos y ponen mayor énfasis en las personas y la comunicación que en seguir estrictamente un proceso. También buscan un equilibrio entre demasiado proceso y ningún proceso.

La filosofía ágil se expresa en el Manifiesto Ágil y se sustenta en el empirismo, cuyos tres pilares son:

- Transparencia: el estado del proceso y del producto es visible.
- Inspección: se revisa frecuentemente lo realizado.
- Adaptación: se realizan cambios a partir de lo aprendido.

Valores ágiles

1. Individuos en interacciones, por sobre procesos y herramientas: es preferible un equipo de personas motivadas con un buen funcionamiento y conocimiento, donde la comunicación sea fluida, pero con herramientas pobres para su trabajo, que tener un equipo de individuos desmotivados o poco funcional con las mejores herramientas y procesos.
2. Software funcionando por sobre documentación extensiva: priorizar el software asegura que se tengan versiones estables, incrementales y mejoradas del software al finalizar cada iteración. Esto permite que a medida que el software crezca en cuanto a funcionalidades, tras cada iteración puede ir mostrándose a los usuarios finales y obtener una retroalimentación de ellos. Solo se documenta aquello que agregue valor al producto y esté centrado en el cliente (valor agregado).
3. Colaboración con el cliente por sobre negociación contractual: esto permitirá no sólo tener una mejor noción de aquellos requerimientos de negocio que necesita el cliente, sino también que el equipo ágil se integre a todas las partes interesadas en el software, para que en conjunto puedan trabajar y lograr los mismos objetivos.
4. Responder a cambios por sobre seguir un plan: esto se plantea debido a que es imposible que los usuarios sepan con detalle y certeza todas las funcionalidades que necesitan, y a su vez, con el paso del tiempo surgirán nuevas necesidades. Un equipo ágil está abierto a los cambios, para poder responder ante ellos de una manera rápida y eficiente entregando el mayor valor posible para el cliente y los usuarios, en lugar de apegarse y seguir un plan definido confeccionado en etapas tempranas de requerimientos.

Principios del manifiesto ágil

1. Nuestra mayor prioridad es satisfacer al cliente.
2. Aceptar que los requisitos cambien, incluso en etapas tardías de desarrollo.
3. Entregar software funcional frecuentemente.
4. Técnicos y no técnicos trabajando juntos durante todo el proyecto (responsables de negocio, desarrolladores, diseñadores).
5. Desarrollamos proyectos en torno a individuos motivados.
6. El método más eficiente de comunicar información es con conversaciones cara a cara.

7. La mejor métrica de progreso es el software funcionando.
8. Los procesos ágiles promueven el desarrollo sostenible.
9. La atención continua a la excelencia técnica y al buen diseño mejora la agilidad.
10. La simplicidad es esencial.
11. Las mejores arquitecturas, diseños y requerimientos surgen de equipos autoorganizados.
12. A intervalos regulares, el equipo evalúa su desempeño y ajusta la manera de trabajar.

Requerimientos en ambientes ágiles - User Stories

Lo más difícil del desarrollo de software es determinar qué software construir, no la tecnología. Muchos proyectos fracasan porque lo desarrollado no coincide con lo que el cliente necesitaba o esperaba, un problema relacionado con la crisis del software.

Una causa importante es subestimar el relevamiento de requerimientos, que es principalmente una actividad social y comunicacional, no técnica.

También es fundamental la comunicación entre clientes y desarrolladores:

- Si domina el cliente, puede pedir funcionalidades o plazos imposibles de cumplir.
- Si dominan los desarrolladores, la jerga técnica puede dificultar entender las necesidades reales del negocio.
- Lo ideal es una comunicación equilibrada entre ambas partes.

Las entregas tempranas permiten al usuario detectar errores, cambiar requisitos o generar nuevas ideas antes de que el proyecto avance demasiado.

Por esto, no es posible planificar perfectamente todo un proyecto desde el inicio mediante herramientas como PERT, ya que al comienzo no se conocen todas las actividades y necesidades futuras.

Fundamentación de los requerimientos ágiles

Los requerimientos ágiles se basan en los valores del Manifiesto Ágil, especialmente en la entrega temprana y frecuente de software útil y en la posibilidad de aceptar cambios incluso en etapas avanzadas.

Se apoyan en 3 pilares:

- Construir el producto correcto: el objetivo es generar valor para el cliente y el negocio, desarrollando software que realmente facilite el trabajo de sus usuarios.
- Determinar "solo lo suficiente": no es necesario conocer todos los requerimientos desde el principio. Se comienza con lo mínimo necesario, se obtiene retroalimentación y se van descubriendo y ajustando nuevos requerimientos.
- Usar historias y modelos para definir qué construir: cliente y desarrolladores trabajan juntos, utilizando herramientas como las User Stories para descubrir y comunicar qué producto se necesita. La comunicación directa es fundamental.

Por lo tanto, el relevamiento de requerimientos se convierte en un proceso colaborativo y continuo entre el equipo técnico y el cliente.

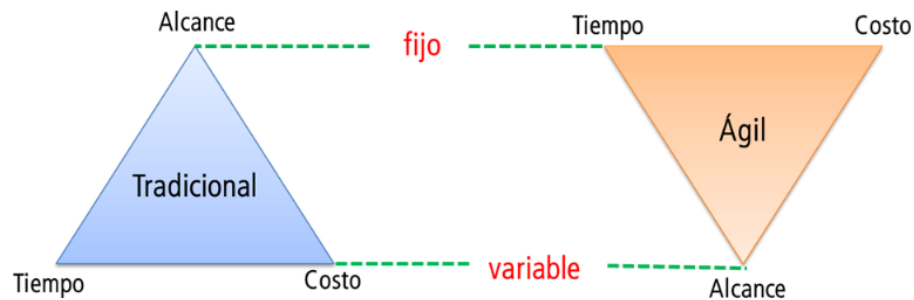
Finalmente, la ingeniería de requerimientos tiene dos procesos:

- Desarrollo de requerimientos: identificar, especificar y validar qué se necesita.

- Gestión de requerimientos: mantenerlos identificados, trazables y actualizados cuando cambian.

La triple restricción - Enfoque ágil

En el enfoque ágil, se trata de aceptar la realidad en la cual el alcance (requerimientos) del producto puede cambiar a lo largo del desarrollo, y no es posible definirlos al comienzo de este. Entonces, el enfoque trata de dejar el tiempo y los recursos fijos, teniendo en cuenta “cuánto software” le puedo entregar en tanto tiempo y con tal equipo de desarrollo.



Por otro lado, en el enfoque tradicional, el alcance se fija y en función de eso derivan los recursos (costo) y el tiempo. El cliente desea controlar el alcance, ya que este comprende lo que él realmente necesita. Luego el cliente pregunta por el tiempo y el costo, los cuales son derivados por el equipo. Por esta razón es tan costosa incorporar cambios de requerimientos, ya que se estaría modificando el alcance y como consecuencia los recursos y el tiempo.

Product Backlog

El Product Backlog es una lista priorizada de requerimientos y funcionalidades, definida por el Product Owner, quien conoce las necesidades del negocio y decide qué debe desarrollarse primero según el valor que aporta.

A diferencia del enfoque tradicional, donde el equipo podía terminar decidiendo la prioridad de los requerimientos, en ágil el cliente participa activamente y establece las prioridades, aumentando las posibilidades de construir un producto que realmente necesita.

Además, la participación constante del cliente y la comunicación cara a cara compensan la menor cantidad de documentación formal y detallada.

Fuente de requerimientos

En el enfoque ágil, los requerimientos se obtienen principalmente mediante comunicación cara a cara con el cliente, lo que permite aclarar dudas y entender mejor sus necesidades.

A diferencia del enfoque tradicional, donde se utiliza una ERS (Especificación de Requerimientos de Software) extensa que el cliente revisa, en ágil se reduce la documentación formal y se compensa con la participación constante del cliente.

Esto no significa no documentar ni especificar requerimientos, sino documentar lo que realmente aporta valor, evitando invertir tiempo en una ERS extensa que podría quedar desactualizada o ser incorrecta.

Tipos de requerimientos

En la gestión ágil de requerimientos se distinguen dos dominios:

- Dominio del problema: incluye los requerimientos de negocio y de usuario. Es lo que le interesa al cliente, porque representa sus necesidades y el valor que espera obtener.
- Dominio de la solución: corresponde a cómo el equipo técnico transforma esas necesidades en software ejecutable. No es el foco principal del cliente.

Las User Stories son requerimientos de usuario y están alineadas con un requerimiento de negocio. Las User Stories no sirven para especificar técnicamente el software, sino para identificar las necesidades del usuario.

User Stories

Las User Stories (US) son una técnica ágil para trabajar requerimientos de usuario.

Describen brevemente una funcionalidad que aporta valor a un usuario.

Su función principal es servir como recordatorio para conversar entre el Product Owner y el equipo sobre qué se necesita construir. Por eso, se consideran un mecanismo para diferir una conversación, en lugar de intentar especificar todo desde el principio.

Características principales:

- Son requerimientos de nivel usuario, no especificaciones técnicas.
- Son escritas por el Product Owner.
- Son ítems de planificación y no acompañan al producto durante todo su ciclo de vida.
- Son verticales, porque una historia puede involucrar distintas partes del sistema (interfaz, lógica, base de datos, etc.).
- Son multipropósito: representan una necesidad, describen parcialmente el producto, permiten planificar y sirven como recordatorio de futuras conversaciones.

La idea surge porque los requerimientos son difíciles de definir completamente al inicio debido a la incertidumbre y los cambios. En lugar de tomar todas las decisiones al comienzo, las User Stories permiten posponer decisiones y detallarlas progresivamente, a medida que aumenta la información y disminuye la incertidumbre.

Partes de la User Story

1. Tarjeta: es una descripción de la historia, utilizada para planificar y como recordatorio.

Tiene una sintaxis particular que es la siguiente:

<<Frase Verbal>>
Como << nombre del rol >> yo puedo << actividad >> de forma tal que << valor de negocio >>

- Nombre del rol: representa quien está realizando la acción o quien recibe el valor de la actividad.
- Actividad: representa la acción que realizará el sistema.
- Valor de negocio: es la más importante y representa el por qué es necesaria la actividad. Esto justifica la razón de que el Product Owner escriba las US. Permite priorizar el desarrollo.

- Frase verbal: no es obligatoria, es una forma corta de referenciar la US. Representa la funcionalidad que se expresa en la User Story.
- Criterios de aceptación: son diferentes criterios sobre la actividad que se necesita implementar, que permiten definir los límites de la U.S. para que el equipo tenga una mejor visión de esta. Además, facilitan las tareas de desarrolladores y testers para probar la funcionalidad. Se deben redactar independientes a la implementación (alto nivel de abstracción).
- Pruebas de aceptación: acompañan a la tarjeta en la parte trasera, y es donde se capturan todos los detalles (que se manifiestan en la conversación) de la User, permitiendo determinar cuándo una historia está completa. Esto servirá a los desarrolladores para probar si la implementación ha sido realizada de forma correcta y completa. Las pruebas pueden agregarse o quitarse en cualquier momento.

2. Conversación: Es la parte más importante de la US, ya que explicita la comunicación entre el Product Owner y el Equipo de desarrollo que permite compensar la falta de especificación y detalle. El acuerdo por sobre la negociación del contrato y el principio de técnicos y no técnicos trabajando juntos durante todo el proyecto están asociadas a esta parte de la US.

La conversación se materializa de forma distribuida en la tarjeta y en las pruebas de aceptación. En simples palabras, son discusiones acerca de la historia que sirven para desarrollar los detalles de la historia.

3. Confirmación: Definición de un acuerdo entre el Product Owner y el Equipo para demostrar que la característica del producto realmente se implementó. Conforma a la definición de las pruebas de aceptación. El PO decide si las pruebas de aceptación son válidas.

Tamaño

Es recomendable tener una gran cantidad de historias con tamaños relativamente pequeños, que tener historias muy grandes. El tamaño de las U.S. se mide en Story Points. Hay 3 aspectos que determinan el tamaño de una U.S.:

- Complejidad: hace referencia a una dificultad inherente a la funcionalidad, que no permiten dividirla en U.S. más pequeñas. Por ejemplo, algoritmos complejos, cálculos, gran cantidad de campos, muchas validaciones, demasiados filtros, etc.
- Esfuerzo: es el tiempo en horas de trabajo que requerirá la implementación.
- Incertidumbre
 - Técnica: cuando se tiene dudas en el dominio de la solución, por ejemplo, el uso de una cierta tecnología no muy conocida por el equipo.
 - De negocio: cuando hay incertidumbre respecto a cómo interactuará el usuario con el sistema, lo cual es causado por falta de conocimiento del dominio del negocio.

Story Points

Son unidades de medida relativas (no importa el valor en sí, sino su comparación), las cuales miden el tamaño de un producto, NO de una medida basada en el tiempo (por

ejemplo, horas hombre). Esto se debe a que estimar basándonos en el tiempo, conduce a muchos errores.

Niveles de abstracción

1. User Story: cumple con el criterio de Listo (DoR) para ingresar a una iteración.

2. Épica: es una User muy grande. El tamaño es relativo a si entra en una iteración o no, ya que las User se empiezan y se terminan en la iteración. Si la User no entra en la iteración es considerado una épica. Tienen la misma sintaxis que la User.

Una épica se suele identificar también cuando no cumple con el modelo de calidad de INVEST. La épica debe ser dividida en US más pequeñas.

Sirven para aquellos requerimientos que no son prioridades en las primeras iteraciones, y basta con tener sólo una aproximación general de las funcionalidades requeridas. Llegado el momento de su implementación, se deberán dividir en historias de usuario más pequeñas, para poder planificar su implementación en sucesivas iteraciones.

3. Temas: son un conjunto de User Stories agrupadas, debido a que conceptualmente están relacionadas. Se las agrupa para tratarlas como una entidad simple a la hora de estimar o planificar.

Criterios de aceptación

Es una nota no obligatoria que define un acuerdo respecto a cómo se debe comportar el software para que el PO acepte la implementación. Lo fundamental es que esté escrito en términos objetivos y concretos.

- Definen límites para una User Story (US)
- Ayudan a que los PO respondan lo que necesitan para que la US provea valor (requerimientos funcionales mínimos).
- Ayudan a que el equipo tenga una visión compartida de la US.
- Ayudan a desarrolladores y testers a derivar las pruebas.
- Ayudan a los desarrolladores a saber cuándo parar de agregar funcionalidad en una US.

¿Cuáles son los criterios de aceptación buenos?

- Definen una intención, no una solución. Ejemplo: El usuario debe elegir al menos una cuenta para operar.
- Son independientes de la implementación.
- Relativamente de alto nivel, no es necesario que se escriba cada detalle

Pruebas de aceptación

Son declaraciones de intención de que hay que probar. El conocimiento del negocio está inmerso en las pruebas de aceptación. Se prueban escenarios exitosos y escenarios que fallen. En las pruebas de aceptación se encuentran las más importantes, ya que no es posible escribir todas las posibles pruebas de aceptación.

El Product Owner acepta la User como implementada si todas las pruebas de aceptación se cumplen. Expresan detalles resultantes de las conversaciones entre clientes y desarrolladores; suelen usarse para completar detalles de la US. Es importante mencionar

que las mismas deben escribirse antes que la programación empiece, y que se recomienda que las escriba el cliente.

DoR (Definition of Ready)

DoR (Definition of Ready): determina si una User Story está preparada para entrar en una iteración. Como mínimo debe cumplir INVEST, aunque el equipo y el Product Owner pueden agregar condiciones.

INVEST

I – Independiente: puede desarrollarse y priorizarse sin depender de otra US.

N – Negociable: describe qué necesita el usuario, no cómo implementarlo. Los detalles se acuerdan después.

V – Valuable: debe aportar valor al usuario o al negocio, no simplemente al equipo técnico.

E – Estimable: debe existir suficiente información para estimar su esfuerzo. Si no se puede estimar, puede deberse a incertidumbre del dominio, incertidumbre técnica o a que la US es demasiado grande.

S – Small (Pequeña): debe poder completarse dentro de una iteración.

T – Testable: debe poder comprobarse mediante criterios/pruebas de aceptación.

DoD (Definition of Done)

Determina cuándo una User Story está realmente terminada. Es un conjunto de condiciones o checklist que debe cumplir para considerarse completa, demostrable y potencialmente desplegable.

Diferencia clave:

DoR = ¿Está lista para empezar?

DoD = ¿Está terminada correctamente?

Spikes

Una Spike es un trabajo especial utilizado para reducir incertidumbre o riesgo cuando una User Story no puede estimarse adecuadamente.

Sirve para investigar, experimentar o hacer prototipos y, una vez resuelta la incertidumbre, la Spike puede dar lugar a una o más User Stories.

Hay dos tipos principales:

- Spike técnica: resuelve incertidumbres sobre tecnología o implementación. Depende principalmente del equipo técnico.
- Spike funcional: resuelve incertidumbres sobre el comportamiento del sistema y la interacción con el usuario. Depende principalmente del negocio/Product Owner.

Las Spikes deberían ser estimables, demostrables y aceptables, y utilizarse como excepción, reservándolas para incertidumbres importantes. Generalmente se realizan en una iteración separada de las historias que surgen como resultado.

Estimaciones en ambientes ágiles

Las estimaciones ágiles son aproximaciones que ayudan a determinar cuánto trabajo puede realizar un equipo. No son compromisos ni predicciones exactas, y se van ajustando a medida que aumenta la información.

¿Quién estima?

El equipo de desarrollo, de forma colectiva. El Product Owner no estima. La idea es que quienes realizan el trabajo sean quienes estimen, aprovechando el conocimiento de todos.

¿Qué se estima?

Se estima el tamaño de una User Story, no su duración en horas. Para determinarlo se consideran:

- Complejidad: qué tan difícil es implementar la US.
- Esfuerzo: cantidad de trabajo necesario.
- Incertidumbre: cuánto se desconoce sobre el problema o la solución.

Estos tres factores forman el Story Point, que es una medida relativa. Por ejemplo, una US de 8 puntos es más grande que una de 5 para ese equipo, pero los puntos no representan horas.

Se suelen utilizar escalas como Fibonacci: 1, 2, 3, 5, 8, 13..., porque la complejidad aumenta de manera no lineal.

Velocidad

La velocidad no se estima, se mide. Es la cantidad de Story Points de las User Stories que el equipo termina completamente y que el PO acepta durante una iteración. Si una US queda a medias, no se cuentan sus puntos.

Se utiliza el promedio de velocidades de varias iteraciones para obtener una referencia más estable del ritmo del equipo.

Reestimación

Una US se reestima cuando cambia la percepción de su tamaño relativo respecto de las demás. No se debería reestimar simplemente porque el equipo tuvo una velocidad menor a la esperada.

Planning Poker

Es una técnica de estimación grupal:

1. Se elige una US de referencia o canónica.
2. Cada integrante estima individualmente usando cartas de Story Points.
3. Todos muestran su estimación simultáneamente.
4. Si hay diferencias importantes, los valores extremos explican sus razones.
5. Se discute y se vuelve a estimar hasta alcanzar una estimación consensuada.

El PO puede participar como moderador y responder dudas, pero no realiza la estimación.

Planificación ágil

Se realiza en distintos niveles:

- Release: determina qué funcionalidades entrarán en una entrega y se actualiza durante el proyecto.

- Iteración/Sprint: selecciona las User Stories que se intentarán completar en la próxima iteración.
- Día: coordina las tareas del equipo mediante reuniones diarias.

Agile vs Tradicional

Agile	Tradicional
Tiempo fijo (Sprint)	Alcance fijo
Alcance variable	Producto definido desde el inicio
Estimaciones relativas	Estimaciones absolutas
Se estima con Story Points	Se suele estimar en tiempo/costo
Estima el equipo	Puede estimar un líder/experto
Busca una precisión suficiente	Busca mayor precisión
Las estimaciones no son compromisos	Pueden utilizarse como base de planificación/compromiso
Se ajusta el alcance	Se intenta mantener el alcance

Framework SCRUM

Scrum es un marco de trabajo ágil para generar valor en problemas complejos mediante un enfoque iterativo e incremental. Se basa en empirismo y pensamiento Lean.

La idea central es:

Product Owner prioriza → equipo desarrolla en un Sprint → se inspecciona → se adapta → se repite.

Pilares de Scrum

- Transparencia: el trabajo y su estado deben ser visibles y comprensibles.
- Inspección: se revisa frecuentemente el trabajo y el progreso.
- Adaptación: si se detectan problemas o cambios, se ajusta el proceso o el producto rápidamente.

Valores

Compromiso – Enfoque – Apertura – Respeto – Coraje.

Scrum Team

Es un equipo pequeño, multifuncional y autogestionado, sin jerarquías ni subequipos. Está compuesto por:

1. Product Owner: Maximiza el valor del producto, gestiona el Product Backlog, define y comunica el Product Goal, ordena/prioriza los elementos del Product Backlog. Es una sola persona, no un comité.

2. Developers: Construyen el Incremento, crean el Sprint Backlog, mantienen la calidad según la Definition of Done, adaptan diariamente su trabajo hacia el Sprint Goal.
3. Scrum Master: Ayuda a que Scrum sea comprendido y aplicado correctamente, facilita la autogestión y mejora del equipo, ayuda a eliminar impedimentos, facilita los eventos de Scrum.

Sprint

Es el contenedor de todos los eventos de Scrum y tiene una duración fija de un mes o menos. El objetivo es producir un incremento de valor. Dentro del Sprint ocurren:

- Sprint Planning
- Daily Scrum
- Sprint Review
- Sprint Retrospective

Durante el Sprint:

- No se realizan cambios que pongan en peligro el Sprint Goal.
- La calidad no disminuye.
- El alcance puede aclararse o renegociarse.
- El Product Backlog puede refinarse.

Eventos

- Sprint Planning: se decide por qué es valioso el Sprint, qué se hará y cómo se realizará.
- Daily Scrum: reunión de máximo 15 minutos para inspeccionar el progreso y adaptar el plan. Es principalmente para los Developers.
- Sprint Review: se inspecciona el resultado del Sprint junto con los stakeholders y se analiza qué hacer a continuación.
- Sprint Retrospective: el equipo analiza cómo trabajó y busca mejorar su calidad y eficacia.
- Refinamiento: actividad continua donde se descomponen, detallan, ordenan y estiman elementos del Product Backlog.

Artefactos de Scrum

Hay 3 artefactos principales, cada uno con un compromiso:

Artefacto	¿Qué contiene?	Compromiso
Product Backlog	Todo el trabajo necesario para mejorar el producto	Product Goal
Sprint Backlog	Objetivo + elementos seleccionados + plan	Sprint Goal
Incremento	Producto funcional generado durante el Sprint	Definition of Done

Product Backlog

Es una lista ordenada y emergente de todo lo que se necesita para mejorar el producto. Es la única fuente de trabajo del Scrum Team.

Sprint Backlog

Es el plan de los Developers para alcanzar el Sprint Goal. Contiene:

- Por qué → Sprint Goal
- Qué → elementos seleccionados del Product Backlog
- Cómo → plan para construir el Incremento

Incremento

Es una parte funcional y utilizable del producto que contribuye al Product Goal. Puede haber varios incrementos dentro de un Sprint. Para ser considerado Incremento, debe cumplir la Definition of Done.

Capacidad de un equipo en un Sprint

La capacidad indica cuánto trabajo puede asumir el equipo durante un Sprint.

- Se estima y se mide.
- Se determina durante el Sprint Planning.
- Permite decidir cuántas User Stories (US) del Product Backlog entran en el Sprint Backlog.
- Equipos maduros → pueden estimarla en Story Points.
- Equipos menos maduros → pueden estimarla en horas ideales.

Niveles de planificación

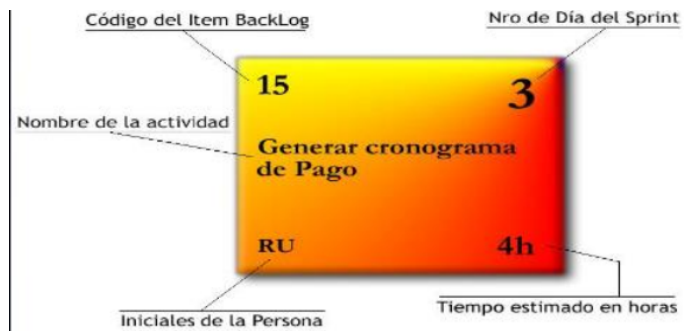
Nivel	Horizonte	Quién	Foco	Entregable
Portfolio	1 año o más	Stakeholders y PO	Administración de un portfolio de producto	Backlog de portfolio
Producto	Arriba de varios meses o más	Stakeholders y PO	Visión y evolución del producto a través del tiempo	Visión del producto, roadmap y características
Release	3 a 9 meses	Scrum team entero y Stakeholders	Balancear continuamente el valor del cliente y la calidad global con las restricciones de alcance, cronograma y presupuesto	Plan de release
Iteración	Cada iteración	Scrum Team entero	Qué aspectos entregar en el siguiente sprint	Objetivo del sprint y sprint backlog
Día	Diaria	SM y desarrolladores	Cómo completar lo comprometido	Inspección del progreso

Herramientas de SCRUM

Tarjeta de tarea

Cada tarea tiene:

- Código del ítem del Backlog.
- Nombre de la actividad.
- Iniciales del responsable.
- Día del Sprint.
- Tiempo estimado para terminarla.



Tablero

Muestra visualmente el estado de las tareas:

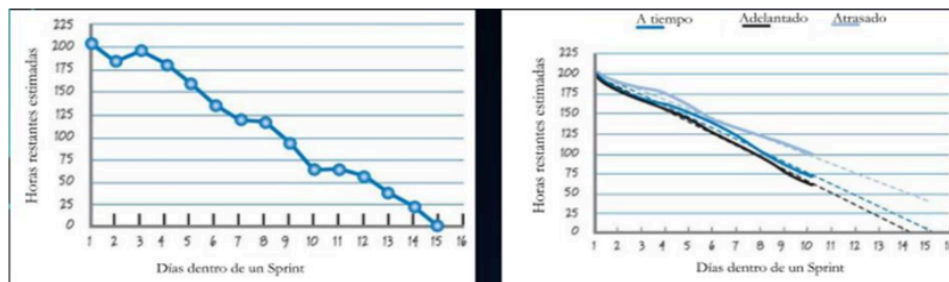
- To Do: tareas pendientes.
- Work In Process: tareas en ejecución.
- To Verify: terminadas, esperando verificación.
- Done: terminadas y verificadas.

Gráficos del Backlog

Sirven para visualizar el progreso de un Sprint, una release o el producto.

Sprint Burn-Down

Muestra cuánto trabajo queda por hacer.



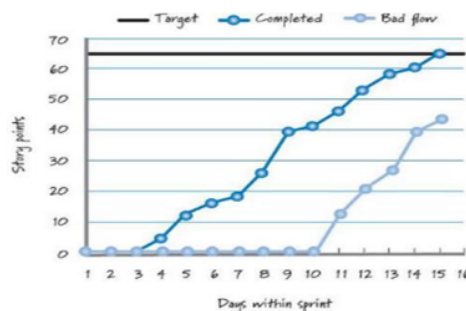
Eje X → tiempo del Sprint.

Eje Y → trabajo restante (Story Points).

La curva debería ser descendente.

Sprint Burn-Up

Muestra cuánto trabajo se completó.



La curva es ascendente.

Permite ver los Story Points terminados a lo largo de los Sprints de una release.

Combined Burn Chart

Visualiza cuánto trabajo queda para terminar y cuánto trabajo se ha realizado.

Unidad 3 - Gestión de configuración de Software - SCM

La Gestión de Configuración es una disciplina de soporte que busca mantener la integridad del software durante todo su ciclo de vida. Como el software cambia constantemente, se generan diferentes versiones, por lo que es necesario controlar y registrar qué cambió, cuándo, por qué y quién lo hizo, manteniendo la trazabilidad entre los cambios y los requisitos.

Su objetivo es identificar y documentar los elementos del software, controlar sus cambios y verificar que cada modificación mantenga la correspondencia con los requerimientos. De esta manera, evita problemas como la pérdida de componentes o cambios, trabajar con versiones desactualizadas, regresiones de errores, superposición de cambios o incorporación de modificaciones no validadas.

Se considera que un producto mantiene su integridad cuando satisface las necesidades del usuario, permite rastrear los cambios durante todo su ciclo de vida, cumple con los requisitos funcionales y no funcionales, y se mantiene dentro de las expectativas de costo. La trazabilidad permite saber qué elementos pueden verse afectados cuando se modifica un requisito o componente, aunque mantener mayor trazabilidad también implica mayor costo. La gestión de configuración es responsabilidad de todo el equipo, aunque puede existir un Gestor de Configuración encargado de tareas específicas, como mantener las herramientas y establecer líneas base. En resumen, busca que el software pueda evolucionar mediante cambios controlados, registrados y trazables, evitando perder o sobrescribir trabajo y asegurando la integridad del producto.

Ítem de configuración

Un Ítem de Configuración (IC) es cualquier artefacto del proyecto o producto que necesita ser identificado, almacenado y controlado durante su evolución, como código fuente, documentos, requisitos, arquitectura, casos de prueba, datos, etc.

Versión

Una versión es un estado particular de un ítem de configuración en un momento determinado. Cada versión se identifica de forma única y representa una etapa en la evolución de ese ítem. El control de versiones permite gestionar esta evolución de manera independiente para cada ítem.

Variante

Una variante es una versión que evoluciona por separado para representar una configuración alternativa. Se utiliza, por ejemplo, cuando el mismo software debe funcionar en distintos sistemas operativos o hardware. En la práctica, se relaciona con los branches.

Configuración

Una configuración es el conjunto de todos los ítems de configuración junto con sus versiones específicas en un momento determinado. Es, básicamente, una “foto” del estado completo del producto en ese instante.

Repositorio

Un repositorio es el lugar donde se almacenan los ítems de configuración y se conserva su historial, atributos y relaciones. También permite analizar el impacto de los cambios. Puede ser centralizado, donde un servidor contiene todo el repositorio y ofrece mayor control, o descentralizado, donde cada usuario posee una copia completa del repositorio, ofreciendo mayor disponibilidad y flexibilidad, aunque con un control más complejo.

Release

Un Release es una entrega del sistema que se libera para que sea utilizada por los clientes u otros usuarios de la organización.

Ramas

Una rama (Branch) es una bifurcación del desarrollo que contiene sus propios ítems de configuración y versiones. Permite trabajar de manera independiente, por ejemplo, para experimentar, corregir errores o adaptar el software a distintas plataformas.

La integración de ramas (Merge) ocurre cuando se combinan los cambios de dos ramas, incorporando sus distintas versiones de los ítems de configuración. Una buena práctica es mantener la rama principal (Main) como versión estable y fusionar allí los cambios que hayan sido desarrollados y validados en otras ramas.

Líneas base

Una línea base (Baseline) es un conjunto de ítems de configuración que fueron revisados y aprobados formalmente, por lo que representan un estado estable y sirven como referencia para el desarrollo posterior. Se identifica mediante un tag y se establecen previamente los criterios que deben cumplir los ítems para formar parte de ella.

Una vez establecida, cualquier cambio sobre la línea base debe pasar por un proceso formal de control de cambios, generalmente gestionado por el Comité de Control de Cambios. Si el cambio se aprueba, se actualiza la línea base y se comunica al equipo.

La línea base sirve principalmente como punto de referencia, permitiendo conocer el estado estable del proyecto en un momento determinado, hacer rollback, saber qué versión se llevó a producción y reproducir un estado anterior del desarrollo.

Existen dos tipos principales: las líneas base operacionales, que contienen una versión ejecutable del producto que ya pasó los controles de calidad y cuya primera versión corresponde al primer Release; y las líneas base de especificación o documentación, que contienen elementos como los requisitos y el diseño, y generalmente aparecen antes de contar con código ejecutable.

Planificación de la gestión de configuración de software

La planificación de la Gestión de Configuración de Software (SCM) debe realizarse al inicio del proyecto y establece cómo se van a identificar, almacenar y controlar los elementos del software durante su evolución. El plan define aspectos como las reglas de nombrado de los ítems de configuración, las herramientas a utilizar, los roles y miembros del Comité de Control de Cambios, los procedimientos para solicitar y aprobar cambios, las auditorías, la estructura del repositorio y los registros que deben conservarse.

Esta planificación debe hacerse tempranamente, definiendo desde el comienzo qué documentos y productos del proceso serán administrados. En general, todos los productos generados durante el desarrollo deben estar sujetos a la gestión de configuración.

Actividades relacionadas a la gestión de configuración

Las actividades de gestión de configuración buscan identificar, controlar y mantener la integridad de los ítems de configuración durante el desarrollo. Se aplican tanto en metodologías tradicionales como ágiles, aunque en una metodología ágil pura las auditorías externas no suelen utilizarse.

Identificación de ítems de configuración

La identificación de ítems de configuración consiste en asignar una identificación única a cada elemento y establecer reglas de nombrado, versionado y ubicación dentro del repositorio. También se define qué elementos forman parte de una línea base. Los ítems pueden clasificarse según su duración en ítems de producto, que existen mientras exista el producto; ítems de proyecto, que duran durante el proyecto; e ítems de iteración, que existen durante una iteración específica.

Control de cambios

El control de cambios se aplica principalmente sobre los ítems que forman parte de una línea base. Si se propone modificarla, el Comité de Control de Cambios analiza el impacto del cambio, considerando el esfuerzo, los recursos, posibles efectos secundarios y el impacto sobre la funcionalidad y arquitectura. Si se aprueba, se genera una orden de cambio, se realiza la modificación y luego el comité revisa y aprueba la nueva línea base. Finalmente, se informa a todos los interesados. Esto permite mantener la trazabilidad y conocer qué elementos son afectados por cada cambio.

Auditorías de configuración

Las auditorías de configuración son controles realizados por un auditor independiente y objetivo sobre una línea base, para verificar que se cumplan las reglas establecidas en el plan de gestión de configuración. La auditoría física verifica aspectos como la ubicación de los ítems, el nombrado y la estructura del repositorio. Si esta es correcta, se puede realizar la auditoría funcional, que verifica que la funcionalidad y el rendimiento del software sean coherentes con los requisitos. La primera realiza principalmente verificación, mientras que la segunda realiza validación.

La trazabilidad de requerimientos establece relaciones entre los distintos elementos del desarrollo, por ejemplo, entre un requisito, su diseño, el código que lo implementa y sus casos de prueba. Permite analizar el impacto de un cambio y debería ser bidireccional, es decir, poder seguir la relación desde el requisito hasta el código y también desde el código hasta el requisito.

En metodologías ágiles, muchas de estas actividades se mantienen mediante herramientas como la integración continua (CI), repositorios y reglas de versionado. El control formal mediante un comité y las auditorías pueden reducirse u omitirse durante el sprint, dependiendo de la metodología.

La gestión de configuración permite reducir retrabajo, mejorar la calidad, aumentar la visibilidad del proyecto y mantener la trazabilidad. Sin embargo, puede generar resistencia porque se percibe como burocrática y su implementación puede ser difícil si no existe compromiso del equipo.

Informe de estado

Finalmente, el informe de estado registra cómo evoluciona el sistema respecto de las líneas base y permite que todos trabajen sobre la versión correcta. Incluye información sobre los cambios realizados y permite saber, por ejemplo, cuál es el estado de un ítem, si un cambio fue aprobado, qué versión implementa ese cambio y qué diferencias existen entre dos versiones.

Gestión de configuración en ambientes ágiles

En ambientes ágiles, la Gestión de Configuración cambia su enfoque: en lugar de utilizarse principalmente para controlar a los desarrolladores, sirve para facilitar el seguimiento, la coordinación y la integridad del producto. Como los equipos ágiles son autoorganizados, las auditorías externas y los comités formales de control de cambios pierden protagonismo. La gestión de configuración se adapta al cambio en lugar de intentar evitarlo y se integra con las demás actividades del equipo.

Continuous Integration

En Agile, la SCM busca automatizar todo lo posible, proporcionar retroalimentación continua sobre la calidad y estabilidad, y ser responsable de todo el equipo. Una práctica fundamental es la Integración Continua (Continuous Integration), donde los desarrolladores integran frecuentemente su código en un repositorio compartido y cada integración es verificada mediante pruebas automatizadas. El objetivo es detectar errores rápidamente y mantener el software siempre en un estado estable y compilable.

Continuous Delivery

La Entrega Continua (Continuous Delivery) va un paso más allá: el software se mantiene en condiciones de ser llevado a producción en cualquier momento. Las pruebas, incluyendo las de aceptación, están automatizadas, pero la decisión final de desplegar queda en manos de una persona, como el Product Owner. Por lo tanto, CI es un prerrequisito de Continuous Delivery.

Continuous Deployment

El Despliegue Continuo (Continuous Deployment) automatiza también la puesta en producción, sin intervención humana. Se utilizan pipelines que ejecutan automáticamente las distintas etapas necesarias para desplegar el software.

Existen distintas estrategias de despliegue. En un Canary Deployment, la nueva versión se libera primero a una pequeña parte de los usuarios y, si funciona correctamente, se extiende progresivamente al resto. En un Blue-Green Deployment, se mantienen dos entornos: uno con la versión actual (Blue) y otro con la nueva (Green); el tráfico se pasa progresivamente al nuevo entorno y el anterior queda disponible para realizar un rollback.

En resumen, CI integra y prueba frecuentemente, Continuous Delivery mantiene el software listo para ser liberado y Continuous Deployment automatiza además su liberación a producción. Las estrategias Canary y Blue-Green permiten realizar esos despliegues reduciendo el impacto y facilitando las reversiones.